

PracticePro Voice Dictation System — Implementation Prompt

Paste this directly into z.ai. It is self-contained.

OBJECTIVE

Replace the existing voice note stack (`SaveToNoteForm.tsx`, `ClientIntakeRecorder.tsx`) with a WhisperFlow-style real-time dictation system. Every recording produces **two persisted outputs**:

1. **RAW** — verbatim transcript, no formatting, no cleanup, no hallucination-risk edits. This is the source of truth.
2. **CLEANED** — the raw transcript passed through an AI formatting pass (punctuation, paragraphing, filler-word removal, legal capitalization conventions, Nigerian legal terminology correction). Never overwrites RAW.

Both versions are stored, both are independently selectable by the user, and both are eligible to be handed to downstream AI tasks (matter summarization, DraftPro insertion, ALOA/ARIA context, search indexing) — the user picks which version feeds which task, with CLEANED as default and RAW as an explicit opt-in per action.

Do not build the AI live-conversation mode — that's ALOA Voice Chat, already exists, out of scope.

DATA MODEL (Convex)

ts

```
// convex/schema.ts - new table
voiceNotes: defineTable({
  matterId: v.optional(v.id("matters")), // null = standalone note
  propertyId: v.optional(v.id("properties")), // Atrium equivalent
  userId: v.id("users"),
  status: v.union(
    v.literal("recording"),
    v.literal("transcribing"),
    v.literal("ready"),
    v.literal("failed")
  ),
  rawText: v.string(),
  cleanedText: v.optional(v.string()),
  cleanedAt: v.optional(v.number()),
  cleanupModel: v.optional(v.string()), // which AI model produced CLEAN
  audioStorageId: v.optional(v.id("_storage")), // original audio, kept for re-
  durationMs: v.number(),
  language: v.optional(v.string()), // detected/declared, e.g. "en-
  tags: v.array(v.string()),
  activePreference: v.union(v.literal("raw"), v.literal("cleaned")), // which o
  createdAt: v.number(),
  updatedAt: v.number(),
})
.index("by_matter", ["matterId"])
.index("by_property", ["propertyId"])
.index("by_user", ["userId"]),
```

Rules:

- `rawText` is written once, immutable after `status: "ready"` except by explicit user manual edit (track `rawEditedAt` if you add manual edit — flag this as a deferred item, don't build it now).
- `cleanedText` can be regenerated any number of times (user hits "re-clean" with a different tone/instruction) — keep only the latest, don't version-history it yet.
- Keep `audioStorageId` for at least re-transcription fallback if RAW transcription quality was bad.

TRANSCRIPTION PROVIDER LAYER

Abstract the provider so Whisper and Gemini are interchangeable. z.ai: build this as a strategy interface, not a hard dependency on one vendor.

ts

```
// src/services/transcription/types.ts
interface TranscriptionProvider {
  name: string;
  transcribeStream(
    audioChunk: Blob,
    sessionId: string,
    onPartial: (text: string) => void,
    onFinal: (text: string) => void
  ): Promise<void>;
  transcribeFile(audioBlob: Blob): Promise<string>; // fallback, non-streaming
}
```

- Primary: Whisper API (large-v3 or equivalent hosted endpoint) for RAW transcription — best raw accuracy, especially Nigerian English/Pidgin.
- Fallback: Gemini (existing key, already wired) if Whisper key absent or fails.
- Provider selection stored in user settings, default Whisper if key present else Gemini.
- Streaming: chunk audio every ~1-2s, send to provider, append partial results client-side with a visible cursor. True word-by-word streaming depends on provider support — if the provider only does chunked-final (most REST Whisper endpoints do), fake the "live" feel by streaming chunk-finals fast (1-2s cadence) rather than promising word-by-word you can't deliver. Flag this constraint explicitly to the user in the UI copy ("Transcribing..." not a token-by-token typewriter unless you're actually using a provider with true partial-result streaming).

CLEANUP PASS (RAW → CLEANED)

Separate Convex action, callable independently of recording (so users can re-clean later, or clean a note that was raw-only from the old system).

```
ts
// convex/voiceNotes.ts
export const cleanTranscript = action({
  args: { voiceNoteId: v.id("voiceNotes"), style: v.optional(v.string()) },
  handler: async (ctx, { voiceNoteId, style }) => {
    // fetch rawText, call AI provider with system prompt below,
    // write cleanedText + cleanedAt + cleanupModel back to the record
  },
});
```

System prompt for cleanup (send RAW verbatim as user content, this as system):

You clean up dictated legal/property notes for a Nigerian legal practice management platform. You will receive a raw, unpunctuated speech-to-text transcript. Return ONLY the cleaned text, nothing else.

Rules:

- Add punctuation, capitalization, paragraph breaks based on natural pauses/topic shifts.
- Remove filler words (um, uh, like, you know) and false starts/stutters.
- Do NOT change, add, or infer any factual content – names, dates, amounts, addresses, legal terms must be preserved exactly as spoken.
- Correct obvious mis-transcriptions of Nigerian legal/property terms (e.g. "affi davit" → "affidavit", "tenency" → "tenancy") but do not correct terms you are not confident about – leave those as-is.
- Preserve spoken voice commands verbatim if any remain unprocessed (e.g. "new paragraph") rather than guessing intent.
- If the speaker is dictating in Nigerian Pidgin, preserve Pidgin phrasing; do not translate to English.
- Never fabricate content that was not in the raw transcript.

Never let CLEANED silently replace RAW in the UI without the user being able to see/toggle back to RAW in one tap.

COMPONENTS

VoiceRecorder.tsx

Replaces the recording UI in `SaveToNoteForm` and `ClientIntakeRecorder`. Props:

```
ts

interface VoiceRecorderProps {
  matterId?: Id<"matters">;
  propertyId?: Id<"properties">;
  onComplete: (voiceNoteId: Id<"voiceNotes">) => void;
  onCancel: () => void;
  mode: "note" | "dictation"; // dictation = DraftPro insert mode, no Convex sa
}
```

- Mic button, waveform/energy visualizer (keep existing `AnalyserNode` pattern).
- Live transcript pane showing RAW as it streams, with a blinking cursor at the end.
- Toggle: "Show cleaned version" (disabled until recording stops + cleanup pass completes).
- Voice command chip feedback (e.g. small toast "→ new paragraph" when a command is detected, so user gets confirmation without breaking flow).

TranscriptionDisplay.tsx

Pure display component, takes `{ rawText, cleanedText, activeView, onToggleView }`. Renders diffing highlight optional (flag as deferred — not needed for v1, plain toggle is enough).

VoiceCommandProcessor.ts (not a component, a pure function/hook)

```
ts

interface VoiceCommand {
  pattern: RegExp;
  action: "new_paragraph" | "new_line" | "insert_placeholder" | "stop" | "cancel_recording";
  extractArg?: (match: RegExpMatchArray) => string; // e.g. placeholder name
}
```

Run against each finalized chunk (not partials — commands only fire on finalized text to avoid false positives on partial hypotheses). Strip the command phrase from the text that gets appended to the transcript buffer.

Command set for v1 (exact phrases, case-insensitive, must appear as a standalone utterance-ish — anchor with word boundaries):

- "new paragraph" → paragraph break
- "new line" → soft break
- "insert [X]" → DraftPro placeholder insert, X mapped against the existing placeholder registry (the color-coded seven-category system already in DraftPro — reuse that lookup, don't build a new one)
- "stop recording" / "stop" → end + finalize
- "cancel recording" / "cancel" → discard, no Convex write

Flag as deferred: fuzzy/synonym matching, multi-language command sets (Pidgin command phrases) — v1 is English exact-phrase only.

HOOKS

```
ts
```

```
// useVoiceRecorder.ts
function useVoiceRecorder(opts: { onChunk: (blob: Blob) => void }): {
  status: "idle" | "requesting-permission" | "recording" | "stopped" | "error";
  error: PermissionError | RecordingError | null;
  start: () => Promise<void>;
  stop: () => void;
  cancel: () => void;
  energyLevel: number; // 0-1, for visualizer
}

// useTranscription.ts
function useTranscription(provider: TranscriptionProvider): {
  rawText: string;
  isTranscribing: boolean;
  pushChunk: (blob: Blob) => void;
  finalize: () => Promise<string>; // returns final raw text
  reset: () => void;
}

// useVoiceCommands.ts
function useVoiceCommands(onCommand: (cmd: VoiceCommand, arg?: string) => void) {
  processChunk: (finalizedText: string) => string; // returns text with command
}

// useCleanup.ts
function useCleanup(): {
  cleanedText: string | null;
  isCleaning: boolean;
  clean: (voiceNoteId: Id<"voiceNotes">, style?: string) => Promise<void>;
  reclean: (voiceNoteId: Id<"voiceNotes">, style?: string) => Promise<void>;
}
```

PERMISSIONS

Web

- Check `navigator.permissions.query({ name: 'microphone' })` where supported (Chrome/Edge; Safari/Firefox don't support this query — fall back to try/catch on `getUserMedia`).
- Require HTTPS — if `window.isSecureContext === false`, show a blocking message, don't attempt `getUserMedia` (it'll throw anyway, but a clear message beats a console error).

APK (Capacitor)

- Do NOT request `RECORD_AUDIO` on app launch. Request only when the user first taps the mic button.
- Before the OS prompt, show an in-app rationale sheet: "PracticePro needs microphone access to record voice notes and dictation. Nothing is recorded until you tap record." — then trigger the native prompt.
- On denial: show "Enable in Settings" with a deep link to app settings ((Capacitor plugin: use `@capacitor/app` or a permissions plugin that supports `openSettings()` — check what's already in the project's `capacitor.config.ts` / installed plugins before adding a new one).
- Manifest: confirm `<uses-permission android:name="android.permission.RECORD_AUDIO" />` is present in `android/app/src/main/AndroidManifest.xml`.

Both

Handle and surface distinctly (don't collapse into one generic error toast):

- Permission denied
 - No microphone device found
 - Microphone in use by another app/tab
 - Network failure mid-transcription (keep the audio locally buffered, retry, don't lose the recording)
-

DRAFTPRO INTEGRATION

- Mic icon in the DraftPro ribbon (TipTap toolbar) → opens `VoiceRecorder` in `mode="dictation"`.
 - In dictation mode: no separate save-to-note step. RAW streams and inserts at the TipTap cursor position live (or in short finalized chunks — same streaming caveat as above). CLEANED is not auto-applied inline; offer a "Clean up this dictation" action after stop that replaces the just-inserted raw text in the editor with the cleaned version (single undo-able TipTap transaction).
 - Voice command "insert [placeholder]" calls the existing placeholder insertion command already used by the color-coded placeholder system — locate and reuse that command, don't duplicate the registry.
-

AI TASK HANDOFF (RAW vs CLEANED selection)

Wherever a voice note feeds another AI action (matter summary, ALOA/ARIA context,

search index), the caller must explicitly pass which version:

```
ts  
  
type VoiceNoteSource = { voiceNoteId: Id<"voiceNotes">; version: "raw" | "clean
```

Default `version: "cleaned"` for anything human-facing or going into a document. Default `version: "raw"` for anything evidentiary/audit-relevant (e.g. if a note is ever referenced as a factual record in a matter, prefer RAW to avoid AI-cleanup drift being mistaken for the client's actual words). Surface this choice in the UI as a small toggle wherever a voice note is attached to a downstream action — don't hide it behind a settings default only.

IMPLEMENTATION SEQUENCE

1. Convex schema (`voiceNotes` table) + basic mutations (create, update status, delete).
2. `useVoiceRecorder` + permission flow (web first, then APK-specific rationale/settings-deeplink).
3. `TranscriptionProvider` interface + Whisper implementation + Gemini fallback.
4. `useTranscription` wired to provider, RAW streaming into `VoiceRecorder` UI, save RAW to Convex on stop.
5. `cleanTranscript` Convex action + system prompt + "Show cleaned version" toggle in `TranscriptionDisplay`.
6. `VoiceCommandProcessor` + `useVoiceCommands`, wire the 5 v1 commands.
7. Retrofit `SaveToNoteForm` / `ClientIntakeRecorder` to use the new stack (mode="note").
8. DraftPro ribbon mic button, mode="dictation", cursor-position insert + placeholder command reuse.
9. AI handoff version-selection UI wherever voice notes attach to matter/document actions.

Deferred (explicitly out of scope for this pass): word-by-word true streaming if provider doesn't support it, manual RAW editing with audit trail, cleaned-text version history, fuzzy/Pidgin voice commands, transcript diffing view.